Self_healing Platform Guide

Tanium Fleet Health, Remediation & Reporting

Generated: 2026-10-01
Orchestrator: blr-kiruthika.strykercorp.com
Path: security-tools-orchestrator/Self_healing/

Table of contents

1. Platform overview
2. Architecture
3. End-to-end execution flow
4. Execution model
5. Tech stack
6. SSH mechanism
7. Monthly calendar
8. Configuration (.env)
9. Operator scripts
10. Configured vs on hold
11. Commands
12. Related docs
13. File catalog (by pipeline phase)
14-15. Excel, playbooks, security, docs

1. Platform overview

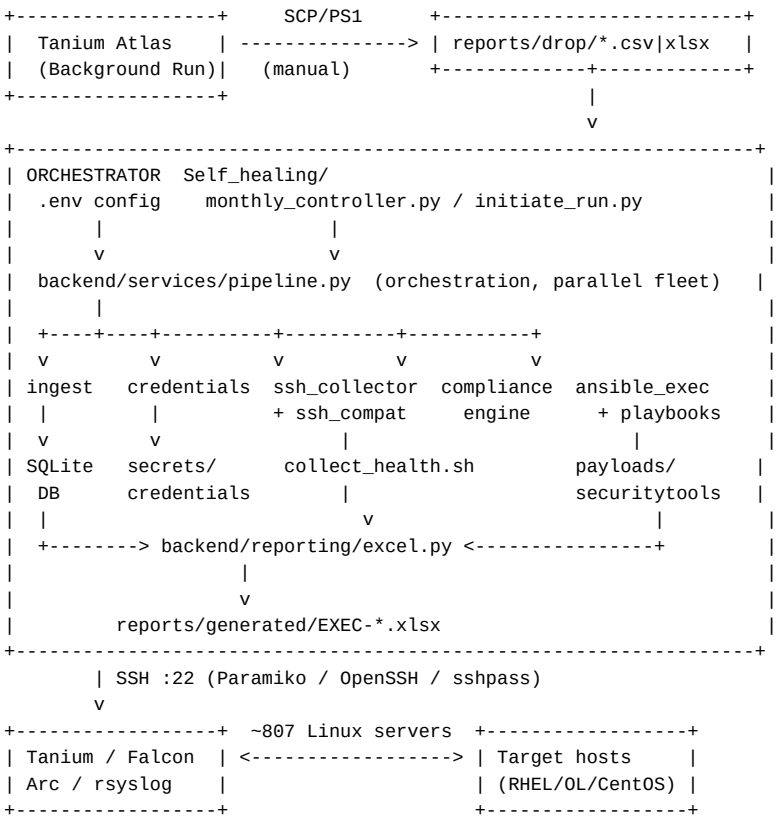
Self_healing is an autonomous security-agent operations platform hosted on the Linux orchestrator (blr-kiruthika.strykercorp.com). It ingests a Tanium Linux fleet export, SSH-connects to each server with up to ten credential profiles, assesses Tanium Client, CrowdStrike Falcon, Azure Arc, and rsyslog health, optionally remediates via Ansible, and produces an Excel execution report.

Primary goal: monthly (15th/29th) closed-loop compliance - inventory in, health out, fixes applied where policy allows, auditable Excel and database history.

Working directory: security-tools-orchestrator/Self_healing/
Manual verify: python3 scripts/verify_readiness.py
Manual run: python3 scripts/initiate_run.py --dry-run|--production --force-rerun

Active Tanium delivery: reports/drop/ (manual SCP). Email Graph and Atlas API fetch are implemented but ON HOLD unless you change TANIUM_REPORT_MODE.

2. Architecture (logical)



3. End-to-end execution flow

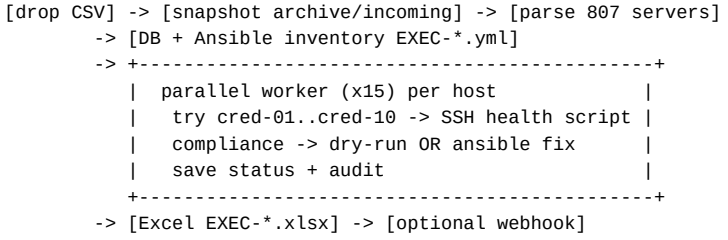
PHASE A - Report arrival (your ops)
  Tanium Background Run produces linux_servers_full_agent_status.csv or .xlsx.
  You upload to reports/drop/ (import_atlas_report.ps1 from Windows or SCP).

PHASE B - Run trigger
  initiate_run.py (manual) OR monthly_controller.py (cron on 15/29).
  snapshot_atlas_export copies drop -> archive + incoming (dated file).

PHASE C - Ingest
  csv_source.py parses rows, extracts IPv4, normalizes OS and optional health columns.
  repository.py upserts Server rows; builder.py writes automation/inventories/EXEC-*.yml.

PHASE D - Fleet processing (parallel workers, default 15)
  For each server: CredentialStore orders up to 10 SSH profiles.
  ssh_collector.py + ssh_compat.py: Paramiko 3.x then OpenSSH/sshpass fallback.
  collect_health.sh runs read-only on target; health.py parses facts.
  compliance/engine.py decides pre-status and remediation actions.
  If DRY_RUN=false: ansible_exec.py runs catalog playbooks with securitytools.env + payloads.
  Post-validation SSH; results persisted; excel.py writes multi-sheet workbook.

PHASE E - Completion
  Execution status: completed | dry_run_completed | safety_paused | invalid.
  notifications/sender.py optional Teams/Slack webhook.



4. Execution model

Concurrency: Thread pool batches of MAX_PARALLEL_SERVERS (default 15). Each worker owns one server end-to-end for that run (SSH -> decision -> remediate -> record).

Idempotency: Processed server IDs skipped on resume. allow_duplicate_execution for forced re-runs via initiate_run.py.

Safety pause: FLEET_FAILURE_THRESHOLD applies to remediation failures after SSH success, not to SSH/auth failures (FLEET_PAUSE_ON_SSH_FAILURES=false). All hosts still receive SSH attempts.

Dry-run: Plans remediations, writes Excel, does not mutate targets (Ansible DRY_RUN).

Credentials: Never stored in Tanium CSV or Excel output - only credential_id labels.

Database: One Execution row per run; many ServerStatus, AgentCheck, Remediation, Audit rows.

5. Tech stack (detailed)

Layer	Technology	Why / need
Language	Python 3.9+	Single orchestrator runtime
Configuration	pydantic-settings, .env	Typed config, no secrets in code
Persistence	SQLAlchemy 2	Executions, audit trail, resume
Database	SQLite / PostgreSQL	Trial vs production scale
Ingestion	csv, openpyxl	Tanium CSV and Excel exports
Scheduling	calendar.py + cron	14/28/15/29 business calendar
SSH client	Paramiko 3.x	Legacy KEX on old hosts
SSH fallback	OpenSSH + sshpass	When Paramiko cannot negotiate
SSH policy	ssh_compat.py	known_hosts, accept-new, scrub
Health probe	Bash collect_health.sh	Standard fact format over SSH
Policy	compliance/engine.py	Map facts to allowed actions
Remediation	ansible-runner	Isolated per-run temp inventory
Automation	21 YAML playbooks	Catalogued install/repair only
Tool config	securitytools.env	Falcon CID, Arc SP, syslog
Packages	RPM payloads on disk	Offline-friendly installs
Reporting	openpyxl	Multi-sheet stakeholder Excel
Optional API	FastAPI	Dashboard and integrations
Optional UI	Next.js	Visual status (not required for CLI)
Optional notify	httpx webhooks	Teams/Slack summary
Optional AI	HTTP LLM	Read-only analysis, policy gated
Legacy bundle	snowcommander/	Optional USE_LEGACY_INSTALLER path

6. SSH mechanism (port 22)

Module backend/services/ssh_compat.py centralizes fleet SSH:

- secrets/known_hosts (project file, accept-new, UpdateHostKeys)
- Auto ssh-keygen -R on host key mismatch then one retry (SSH_AUTO_FIX_HOST_KEYS)
- Legacy KEX: diffie-hellman-group14-sha1, group-exchange-sha1, ssh-rsa, ssh-dss
- RequiredRSASize=768 for RHEL 9 orchestrator to old targets
- OpenSSL legacy config for OpenSSH subprocess
- Paramiko 3.x required (pip install 'paramiko>=3.4,<4')
- sshpass for password authentication when no key

Health collection: automation/scripts/collect_health.sh (sudo -n preferred).
Ansible remediation: same ansible_ssh_common_args on inventory line.

Not fixable in software: firewall blocking TCP 22, sshd down, wrong password, account lockout.

7. Monthly calendar mechanism

```
Day (Asia/Kolkata)     Action
  ----------------       ----------------------------------------
  14, 28                 Snapshot: archive Tanium file from drop
  15, 29                 Full execution pipeline (Feb: 15 only)
  Other days             cron idle (manual initiate_run anytime)

  Retention: REPORT_RETENTION_MONTHS prunes old archive/incoming files
```

16. File catalog - requirement and purpose

A. Report delivery & ingest

reports/drop/linux_servers_full_agent_status.*
  Need: Required
  Does: Tanium fleet inventory source you upload.

scripts/import_atlas_report.ps1
  Need: Ops
  Does: Windows SCP helper into drop folder.

scripts/install_fleet_report.py
  Need: Ops
  Does: Copy report into drop with correct filename on orchestrator.

backend/ingestion/tanium_remote_fetch.py
  Need: Optional
  Does: SFTP/local_share/drop check; idle in drop mode.

backend/ingestion/o365_email_fetch.py
  Need: On hold
  Does: Graph mailbox attachment; not active.

backend/ingestion/atlas_fetch.py
  Need: Required
  Does: Archive drop to incoming + retention.

backend/ingestion/csv_source.py
  Need: Required
  Does: Parse CSV/XLSX into normalized servers.

backend/ingestion/ip_extract.py
  Need: Required
  Does: Extract usable IPv4 from Tanium cells.

backend/ingestion/normalize.py
  Need: Required
  Does: Normalize OS and health column text.

backend/ingestion/archive_retention.py
  Need: Optional
  Does: Prune old archive/incoming files.

B. Run control & schedule

.env
  Need: Required
  Does: Master runtime flags, paths, DRY_RUN, parallelism, SSH, Tanium mode.

backend/config.py
  Need: Required
  Does: Loads environment into Settings object.

scripts/initiate_run.py
  Need: Required
  Does: Manual execution entry point.

scripts/monthly_controller.py
  Need: Required
  Does: Cron entry for calendar snapshot/run.

scripts/install_monthly_cron.sh
  Need: Ops
  Does: Install daily cron invoking controller.

scripts/verify_readiness.py
  Need: Ops
  Does: Pre-flight validation before fleet run.

backend/scheduler/calendar.py
  Need: Required
  Does: Decide run day and report freshness.

backend/models/session.py

Need: Required
Does: Database engine and session factory.

data/self_healing.db
Need: Required
Does: SQLite execution state (or Postgres URL).

C. Pipeline core

backend/services/pipeline.py
Need: Required
Does: End-to-end orchestration, parallel fleet, finalize Excel.

backend/models/entities.py
Need: Required
Does: ORM models for executions and servers.

backend/models/repository.py
Need: Required
Does: Database access, audit log, status persistence.

backend/schemas/domain.py
Need: Required
Does: Ingestion and health data structures.

backend/schemas/status.py
Need: Required
Does: Status enumerations for reporting.

backend/inventory/builder.py
Need: Required
Does: Write Ansible inventory YAML per execution.

D. Credentials & SSH

secrets/credentials/credentials.json
Need: Required
Does: Ten SSH profiles (primary bundle).

backend/credentials/provider.py
Need: Required
Does: Load, order, and resolve credential secrets.

backend/services/ssh_compat.py
Need: Required
Does: Legacy SSH algorithms, known_hosts, Ansible SSH args.

backend/services/ssh_collector.py
Need: Required
Does: SSH health collection with cred rotation.

automation/scripts/collect_health.sh
Need: Required
Does: Remote read-only health fact script.

backend/services/health.py
Need: Required
Does: Parse collector output to HostFacts.

secrets/known_hosts
Need: Auto
Does: Project SSH host key store (accept-new).

config/openssl-legacy-ssh.cnf
Need: Required
Does: OpenSSL legacy for old ssh-rsa via OpenSSH.

E. Compliance, AI, fix

backend/compliance/engine.py
Need: Required
Does: Decide healthy vs actions needed.

backend/remediation/catalog.py
Need: Required
Does: Map actions to playbook files.

backend/remediation/policy.py
  Need: Required
  Does: Validate remediation plan and AI suggestions.

backend/remediation/ansible_exec.py
  Need: Required
  Does: Run Ansible playbooks via ansible-runner.

automation/ansible.cfg
  Need: Required
  Does: Ansible defaults for playbook execution.

secrets/securitytools.env
  Need: For installs
  Does: Falcon, Arc, syslog secrets for remediation.

automation/payloads/tanium/
  Need: For Tanium install
  Does: RPMs and tanium-init.dat.

automation/payloads/crowdstrike/
  Need: For Falcon install
  Does: Sensor RPM packages.

automation/scripts/*_tool.sh
  Need: For installs
  Does: Shell helpers invoked by playbooks.

automation/playbooks/*.yml (21)
  Need: For installs
  Does: Catalogued install/repair/update actions.

backend/ai/analyst.py
  Need: Optional
  Does: Read-only LLM analysis of failures.

backend/vcenter/readonly.py
  Need: Optional
  Does: vCenter correlation if configured.

F. Output & notify

backend/reporting/excel.py
  Need: Required
  Does: Build EXEC-*.xlsx workbook.

reports/generated/EXEC-*.xlsx
  Need: Output
  Does: Stakeholder report per run.

backend/notifications/sender.py
  Need: Optional
  Does: Teams/Slack webhook on completion.

backend/services/redact.py
  Need: Required
  Does: Strip secrets from logs/excerpts.

8. Configuration files (.env highlights)

DATABASE_URL SQLite or PostgreSQL
DRY_RUN true = no installs; false = live remediation on cron/manual production
CREDENTIAL_DIR ./secrets/credentials
TOOL_ENV_FILE ./secrets/securitytools.env
TANIUM_REPORT_MODE drop (manual upload); alternatives: sftp, local_share, o365_email
MAX_PARALLEL_SERVERS 15 recommended for ~807 hosts
MAX_CREDENTIAL_SETS 10 SSH profiles per host
SCHEDULE_TIMEZONE Asia/Kolkata for 14/15/28/29 calendar
SSH_* PORT, KNOWN_HOSTS, LEGACY_ALGORITHMS, ACCEPT_NEW, AUTO_FIX
FLEET_* pause thresholds, progress log interval

9. Operator scripts

verify_readiness.py	Pre-flight checklist before execution
initiate_run.py	Manual full pipeline (dry-run or --production)
monthly_controller.py	Cron entry: fetch (if configured), snapshot, pipeline
install_fleet_report.py	Copy CSV/XLSX into reports/drop/
import_atlas_report.ps1	Windows SCP upload to orchestrator drop folder
install_monthly_cron.sh	Install crontab for monthly_controller
bootstrap_orchestrator.sh	First-time host setup
init_db.py	Create database schema
merge_credentials_to_one_file.py	Build credentials.json from cred-*.json
build_platform_guide_pdf.py	This PDF generator

10. What is configured vs on hold

CONFIGURED NOW:
- ~807-row fleet CSV in reports/drop/
- credentials.json with 10/10 SSH passwords
- securitytools.env, Tanium/CrowdStrike payloads
- TANIUM_REPORT_MODE=drop, DRY_RUN=true (change for production)
- SSH compat layer, fleet-wide processing tuning

ON HOLD (by choice):
- O365 email attachment fetch (fetch_tanium_from_email.py, o365_email mode)
- Tanium Atlas UI / undocumented API automation
- SFTP/local_share until IT provides paths

OPTIONAL / PARTIAL:
- FastAPI dashboard, Next.js UI, vCenter, AI endpoint, Teams/Slack webhooks
- Kubernetes CronJob vs host cron
- USE_LEGACY_INSTALLER snowcommander path

YOUR INTERVENTION EACH CYCLE:
- Upload fresh Tanium export to reports/drop/
- Review dry-run Excel; fix SSH/network/creds
- Set DRY_RUN=false when ready for live remediation

11. Commands quick reference

```
cd /home/tpx-admin/security-tools-orchestrator/Self_healing
python3 scripts/verify_readiness.py
python3 scripts/initiate_run.py --dry-run --force-rerun --parallel 15
python3 scripts/initiate_run.py --production --force-rerun --parallel 15
```

Output: reports/generated/EXEC-YYYY-MM-DD-NNN.xlsx

12. Excel report structure (EXEC-*.xlsx)

Summary	Execution ID, dates, counts (total, healthy, SSH failures, remediation stats)
Agent Compliance	Per-host component health pre/post
Remediation Details	Each action, status, retry count, log excerpt
Failed Servers	Hostname, IP, failure reason (SSH, auth, policy)
Execution Log	Audit trail of pipeline events
AI Analysis	Optional LLM findings if AI_ENDPOINT configured

Purpose: single stakeholder artifact after each run; complements SQLite audit tables.

13. Ansible playbook catalog (21)

Tanium: install_tanium, repair_tanium, remediate_tanium, update_tanium
CrowdStrike: install_crowdstrike, repair_crowdstrike, remediate_crowdstrike, reconfigure_crowdstrike, update_crowdstrike
Arc: install_arc, repair_arc, reconnect_arc, update_arc
Sentinel: install_sentinel, repair_sentinel, restart_sentinel, update_sentinel
rsyslog: install_rsyslog, repair_rsyslog, remediate_rsyslog, update_rsyslog

Each playbook calls matching automation/scripts/*_tool.sh with packages from automation/payloads/ and variables from secrets/securitytools.env.

14. Security and data handling

- Passwords and keys live only under secrets/ (gitignored); never in Tanium CSV or Excel.
- Ephemeral Ansible inventory with passwords is written to temp dir and deleted after run.
- AI and analyst modules are read-only; policy.py blocks disallowed recommendations.
- SSH health uses fixed collect_health.sh only (no arbitrary remote shell API).
- DRY_RUN default true until operator sets false or uses --production explicitly.

15. Related documentation

docs/EXECUTION_OVERVIEW.md	Flow and configuration summary
docs/FINAL_INPUT_CHECKLIST.md	Go-live tick list
docs/CREDENTIALS.md	SSH secret layout
docs/TANIUM_DELIVERY_PATHS.md	Report delivery options
WHERE_EVERYTHING_IS.md	Directory map